

# CHAPTER 12

## 記憶體設計與應用



只要是電路中需要大量儲存資料的地方，不外乎是使用：隨機存取記憶體 (RAM) 或者是唯讀記憶體 (ROM)。

隨機存取記憶體與唯讀記憶體的差異在於：我們可以將資料寫入到隨機存取記憶體中或者是從隨機存取記憶體中讀出資料，但是我們只能從唯讀記憶體中讀出資料。

本章將為各位介紹隨機存取記憶體 (RAM)、唯讀記憶體 (ROM)，它們的應用電路，以及用 RAM 來設計快取記憶體電路。



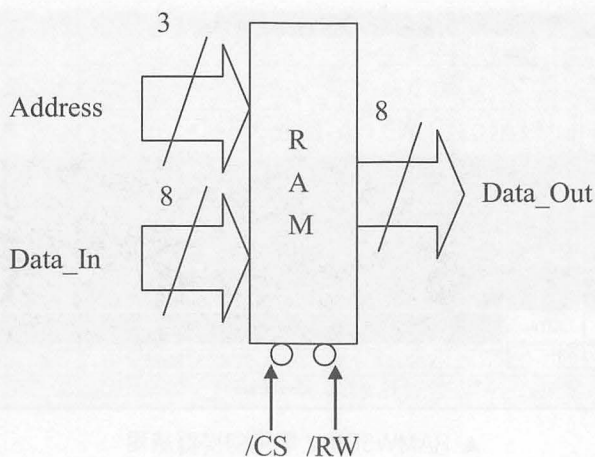
## 12.1 隨機存取記憶體

### (Random Access Memory, RAM)

隨機存取記憶體為一種序向電路，它是由一些基本的記憶單元 (Memory Cell, MC) 與位址解碼器 (Address Decoder) 所組成的。而其內的每一個記憶單元均由一個正反器與一些控制此正反器的資料寫入與讀取動作的邏輯閘電路所組成。

通常隨機存取記憶體都有以下的控制訊號 (如下圖所示)：

- /CS (晶片選擇, Chip Select)
- /RW (讀取/寫入, 0 : Read, 1 : Write)
- Address (位址線)
- Data\_In (資料輸入線)
- Data\_Out (資料輸出線)

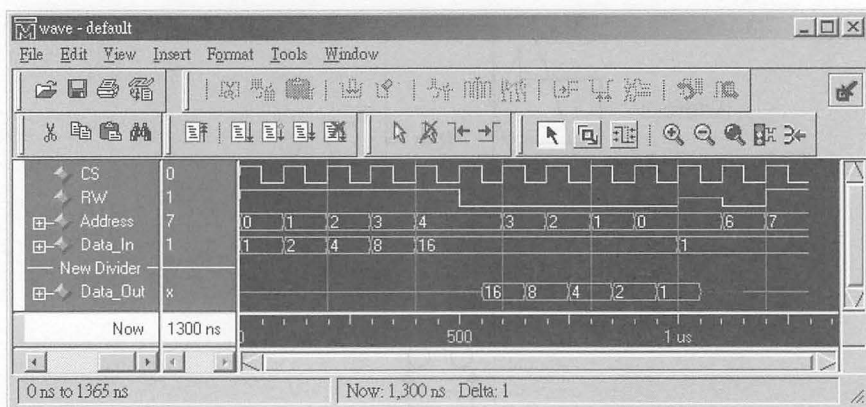


▲ 具有 8 個字組，每個字組有 8 個位元的隨機存取記憶體的邏輯符號

【RAMW5B8.V：具有 5 個字組，每個字組有 8 個位元的記憶體模組】

```
// RAMW5B8.V：具有 5 個字組，每個字組有 8 個位元的記憶體模組
module RAM_Word5_Bit8 (CS, RW, Address, Data_In, Data_Out);
parameter Words = 5;
input CS, RW;
input [2:0] Address;
input [7:0] Data_In;
output [7:0] Data_Out;
reg [7:0] RAM_Data[Words-1:0];
reg [7:0] Data_Out;

always @(negedge CS)
begin // 負緣觸發，選擇到此晶片
    if (RW == 1'b0) // 讀取
        Data_Out = RAM_Data[Address[2:0]];
    else
        if (RW == 1'b1) // 寫入
            RAM_Data[Address[2:0]] = Data_In;
        else
            Data_Out = 8'bz; // 高阻抗 (High Impedance)
    end
endmodule
```



▲ RAMW5B8.V 電路的模擬結果



## 12.2 隨機存取記憶體 (RAM) 的擴充

隨機存取記憶體擴充的方式，分成：位元組 (Bits) 的擴充、字組 (Words) 的擴充這二種方式。

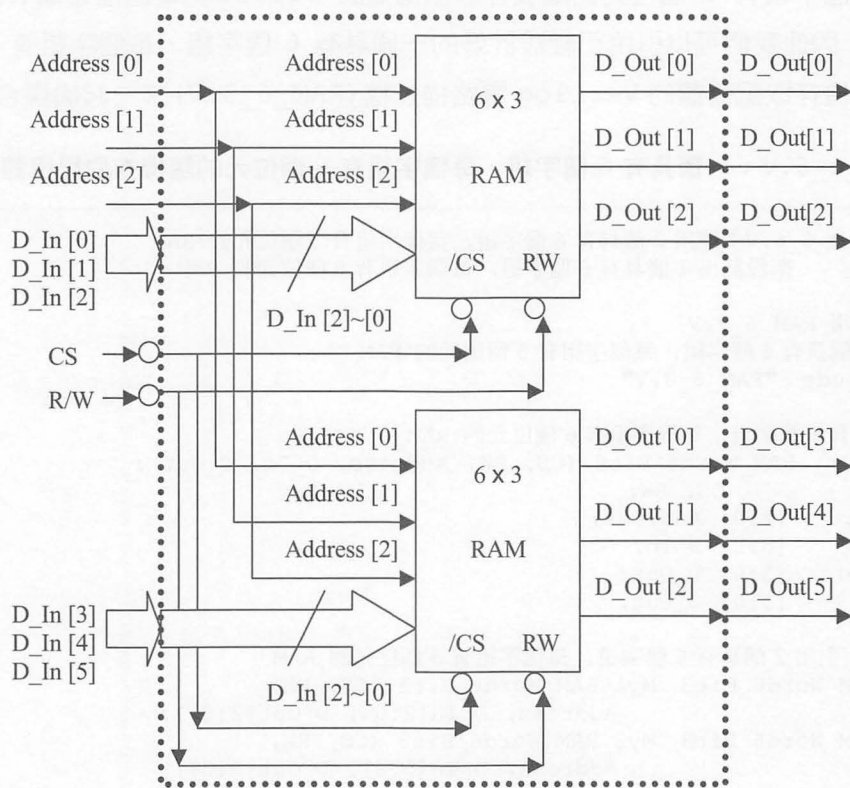
有一個限制條件是：必須要是同樣位元組 (Bits) 大小及字組 (Words) 大小，且具有相同控制訊號功能的 RAM 才可以組成隨機存取記憶體的擴充電路。

### 12.2.1 隨機存取記憶體之位元組 (Bits) 的擴充方式

例如：我們需要設計出一個具有 6 個字組，每個字組有 6 個位元的隨機存取記憶體模組電路 (6x6 RAM)，則必須利用兩個具有 6 個字組，每個字組有 3 個位元的隨機存取記憶體模組 (6x3 RAM)，先將它們所有的晶片選擇線 (Chip Select)、讀寫控制線 (Read/Write) 與位址線 (Address) 全部並聯起來當作是 6x6 RAM 的晶片選擇線 (Chip Select)、讀寫控制線 (Read/Write) 及位址線 (Address)。再將其中一個 6x3 RAM 的資料輸入線 (Data In) 接到 6x6 RAM 資料輸入線 (Data In) 的位元 2 到位元 0 (Data\_In[2:0])，另一個 6x3



RAM 的資料輸入線 (Data In) 接到 6x6 RAM 資料輸入線 (Data In) 的位元 5 到位元 3 (Data\_In [5:3])。最後將其中一個 6x3 RAM 的資料輸出線 (Data Out) 接到 6x6 RAM 資料輸出線 (Data Out) 的位元 2 到位元 0 (Data\_Out [2:0])，另一個 6x3 RAM 的資料輸出線 (Data Out) 接到 6x6 RAM 資料輸出線 (Data Out) 的位元 5 到位元 3 (Data\_Out [5:3])。如圖「由二個 6x3 RAM 組成一個 6x6 RAM 的邏輯符號」。



▲ RAM\_6\_6.V：由二個 6x3 RAM 組成一個 6x6 RAM 的邏輯符號

利用將這兩個 6x3 RAM 的晶片選擇線 (Chip Select) 並聯起來的目的在於：這兩個 6x3 RAM 均同時被選擇到或者是沒有被選擇到，若同時被選擇到且讀寫控制線 (Read/Write) 為 0 時，則這兩個 6x3 RAM 各輸出 3 個位元；若

同時被選擇到且讀寫控制線 (Read/Write) 為 1 時，則輸入的資料分別寫入到這兩個 6x3 RAM 內；因此電路的位元組 (Bits) 數目就結合為 6 個位元了，達到了位元組擴充的目的。

## 範例 RAM\_6\_6.V：位元組 (Bits) 的擴充範例

利用 `include 編譯命令：因為我們要設計的模組功能為一個具有 6 個字組，每個字組有 6 個位元的隨機存取記憶體的 Verilog 電路描述檔 (RAM\_6\_6.V)。因此我們可以引用已經設計好的一個具有 6 個字組，每個字組有 3 個位元的隨機存取記憶體的 Verilog 電路描述檔 (RAM\_6\_3.V) 來一起編譯合成。

### 【RAM\_6\_6.V：一個具有 6 個字組，每個字組有 6 個位元的隨機存取記憶體】

```
// RAM_6_6.V：運用 2 個具有 6 個字組，每個字組有 3 個位元的 RAM
// 來設計出 1 個具有 6 個字組，每個字組有 6 個位元的 RAM

// 引用 RAM_6_3.V
// 一個具有 6 個字組，每個字組有 3 個位元的 RAM
`include "RAM_6_3.V"

// 具有 6 個字組，每個字組有 6 個位元的 RAM
module RAM_Word6_Bit6 (CS, RW, Address, D_In, D_Out);
input CS, RW;
input [2:0] Address;
input [5:0] D_In;
output [5:0] D_Out;
wire [5:0] D_Out;

// 引用 2 個具有 6 個字組，每個字組有 3 個位元的 RAM
RAM_Word6_Bit3 My1_RAM_Word6_Bit3 (CS, RW,
    Address, D_In[2:0], D_Out[2:0]);
RAM_Word6_Bit3 My2_RAM_Word6_Bit3 (CS, RW,
    Address, D_In[5:3], D_Out[5:3]);

endmodule
```

### 【RAM\_6\_3.V：一個具有 6 個字組，每個字組有 3 個位元的隨機存取記憶體】

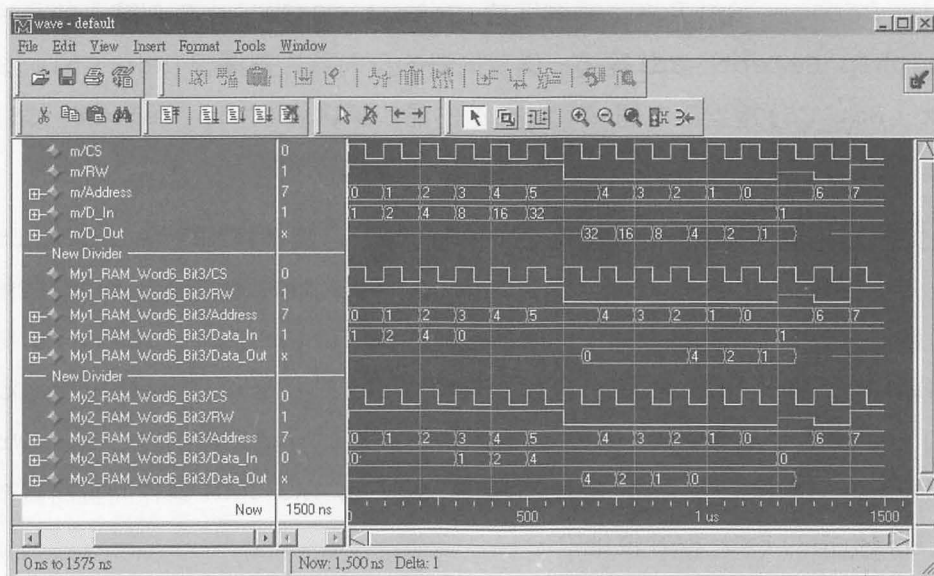
```
// RAM_6_3.V，具有 6 個字組，每個字組有 3 個位元的記憶體模組
module RAM_Word6_Bit3 (CS, RW, Address, Data_In, Data_Out);
```

```

parameter      Words = 6;
input          CS, RW;
input          [2:0] Address;
Input          [2:0] Data_In;
output         [2:0] Data_Out;
reg            [2:0] RAM_Data[Words-1:0];
reg            [2:0] Data_Out;

always @(negedge CS)
begin // 負緣觸發，選擇到此晶片
    if (RW == 1'b0) // 讀取
    begin
        Data_Out = RAM_Data[Address[2:0]];
    end
    else
    begin
        if (RW == 1'b1) // 寫入
        begin
            RAM_Data[Address[2:0]] = Data_In;
        end
        else
            Data_Out = 3'bz; // 高阻抗 (High Impedance)
        end
    end
end
endmodule

```



▲ RAM\_6.V 合成後，電路的模擬結果

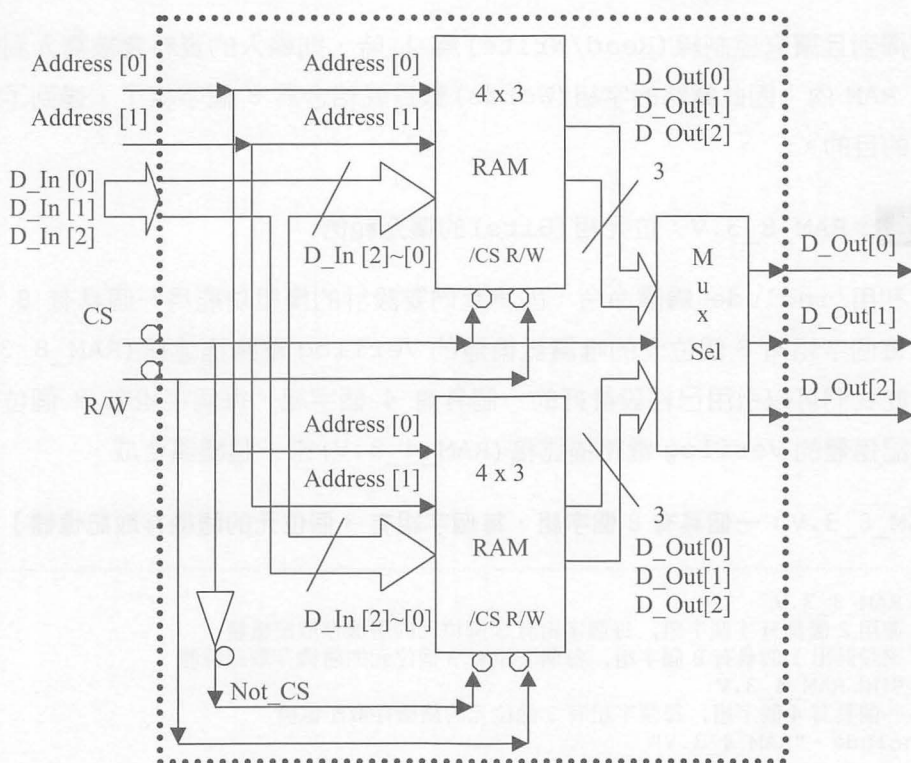
## 12.2.2 隨機存取記憶體之字組 (Words) 的擴充方式

例如：我們需要設計出一個具有 8 個字組，每個字組有 3 個位元的隨機存取記憶體模組電路 (8x3 RAM)，則必須利用兩個具有 4 個字組，每個字組有 3 個位元的隨機存取記憶體模組 (4x3 RAM)，先將它們所有的晶片選擇線 (Chip Select)、讀寫控制線 (Read/Write) 以及位址線 (Address) 全部並聯起來當作是 8x3 RAM 的晶片選擇線 (Chip Select)、讀寫控制線 (Read/Write) 以及位址線 (Address)。

再將其中一個 4x3 RAM 的資料輸入線 (Data In) 接到 8x3 RAM 資料輸入線 (Data In) 的位元 2 到位元 0 (Data\_In[2:0])，另一個 4x3 RAM 的資料輸入線 (Data In) 接到 8x3 RAM 資料輸入線 (Data In) 的位元 2 到位元 0 (Data\_In [2:0])。

最後將它們所有的資料輸出線 (Data Out) 接到多工器的輸入端，而由外部輸入的晶片選擇線 (Chip Select, CS) 接到第一個 4x3 RAM 的晶片選擇線 (Chip Select, CS) 及多工器選擇線 (Select, Sel)，由外部輸入的晶片選擇線 (Chip Select, CS) 經過一個反向器的反向晶片選擇線 (Not Chip Select, Not\_CS) 接到第二個 4x3 RAM 的晶片選擇線 (Chip Select, CS)。如圖「由二個 4x3 RAM 組成一個 8x3 ROM 的邏輯符號」。

如果晶片選擇線 (Chip Select, CS) 為 0 且讀寫控制線 (Read/Write) 為 0 時，則第二個 4x3 RAM 不動作，第一個 4x3 RAM 動作而且將位址線所選到的資料送至多工器的輸入端，同時多工器的選擇線 (Select, Sel) 為 0 時，會將第一個 4x3 RAM 的輸出資料送到整個 8x3 RAM 的資料輸出線 (Data Out)。如果晶片選擇線 (Chip Select, CS) 為 0 且讀寫控制線 (Read/Write) 為 1 時，則第二個 4x3 RAM 不動作，第一個 4x3 RAM 動作而且將資料送至位址線所指的 8x3 RAM 位址中。



▲ RAM\_8\_3.V：由二個 4x3 RAM 組成一一個 8x3 RAM 的邏輯符號

相反的，如果晶片選擇線 (Chip Select, CS) 為 1 且讀寫控制線 (Read/Write) 為 0 時，則第一個 4x3 RAM 不動作，第二個 4x3 RAM 動作而且將位址線所選到的資料送至多工器的輸入端，同時多工器的選擇線 (Select, Sel) 為 1 時，會將第二個 4x3 RAM 的輸出資料送到整個 8x3 RAM 的資料輸出線 (Data Out)。若晶片選擇線 (Chip Select, CS) 為 1 且讀寫控制線 (Read/Write) 為 1 時，則第一個 4x3 RAM 不動作，第二個 4x3 RAM 動作而且將資料送至位址線所指的 8x3 RAM 位址中。

利用將這兩個 4x3 RAM 的晶片選擇線 (Chip Select) 並聯起來的目的在於：這兩個 4x3 RAM 在同時間只有一個會被選擇到，若被選擇到且讀寫控制線 (Read/Write) 為 0 時，則被選擇到的這個 4x3 RAM 輸出 3 個位元；若同時

被選擇到且讀寫控制線 (Read/Write) 為 1 時，則輸入的資料會被寫入到這個 4x3 RAM 內；因此電路的字組 (Words) 數目就結合為 8 個字組了，達到了字組擴充的目的。

### 範例 RAM\_8\_3.V：位元組 (Bits) 的擴充範例

利用 `include 編譯命令：因為我們要設計的模組功能為一個具有 8 個字組，每個字組有 3 個位元的唯讀記憶體的 Verilog 電路描述檔 (RAM\_8\_3.V)，因此我們可以引用已經設計好的一個具有 4 個字組，每個字組有 3 個位元的唯讀記憶體的 Verilog 電路描述檔 (RAM\_4\_3.V) 來一起編譯合成。

【RAM\_8\_3.V：一個具有 8 個字組，每個字組有 3 個位元的隨機存取記憶體】

```
// RAM_8_3.V
// 運用 2 個具有 4 個字組，每個字組有 3 個位元的隨機存取記憶體
// 來設計出 1 個具有 8 個字組，每個字組有 3 個位元的隨機存取記憶體
// 引用 RAM_4_3.V
// 一個具有 4 個字組，每個字組有 3 個位元的隨機存取記憶體
`include "RAM_4_3.V"

// 具有 8 個字組，每個字組有 3 個位元的隨機存取記憶體
module RAM_Word8_Bit3 (RW, CS, Address, D_In, D_Out);
input RW, CS;
input [1:0] Address;
input [2:0] D_In;
output [2:0] D_Out;
reg [2:0] D_Out;
wire Not_CS;
wire [2:0] D_Out_1;
wire [2:0] D_Out_2;

// 作 Invert Chip Select 訊號
assign Not_CS = !CS;

// 引用 2 個具有 4 個字組，每個字組有 3 個位元的隨機存取記憶體
RAM_Word4_Bit3 My1_RAM_Word4_Bit3 (
    CS, RW, Address, D_In, D_Out_1);
RAM_Word4_Bit3 My2_RAM_Word4_Bit3 (
    Not_CS, RW, Address, D_In, D_Out_2);

always @(CS or RW or Not_CS or D_Out_1 or D_Out_2)
```



```

begin
    if (CS == 1'b0) // 第一顆 8x3 RAM
    begin
        D_Out[2:0] = D_Out_1[2:0];
    end
    else
    begin
        if (CS == 1'b1) // 第二顆 8x3 RAM
        begin
            D_Out[2:0] = D_Out_2[2:0];
        end
        else
        begin
            D_Out[2:0] = 3'bz; // 高阻抗 (High Impedance)
        end
    end
end
endmodule
    
```

【RAM\_4\_3.V：具有 4 個字組，每個字組有 3 個位元的隨機存取記憶體】

```

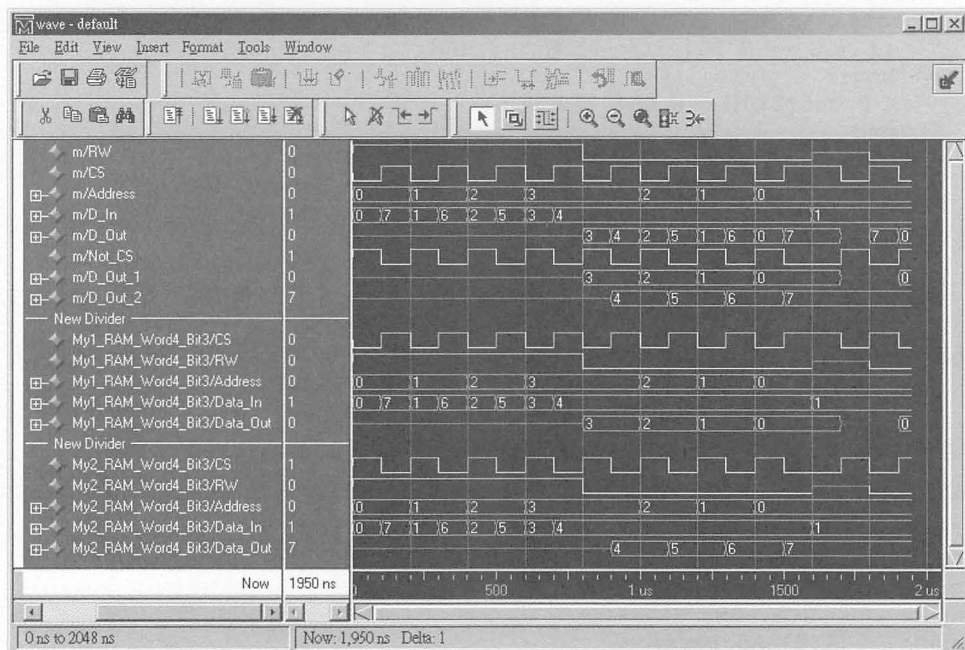
// RAM_4_3.V：具有 4 個字組，每個字組有個 3 位元的記憶體模組
module RAM_Word4_Bit3 (CS, RW, Address, Data_In, Data_Out);
parameter Words = 4;
input CS, RW;
input [1:0] Address;
input [2:0] Data_In;
output [2:0] Data_Out;
reg [2:0] RAM_Data[Words-1:0];
reg [2:0] Data_Out;
always @(negedge CS)
begin // 負緣觸發，選擇到此晶片
    if (RW == 1'b0) // 讀取
    begin
        Data_Out = RAM_Data[Address[1:0]];
    end
    else
    begin
        if (RW == 1'b1) // 寫入
        begin

        begin
            RAM_Data[Address[1:0]] = Data_In;
        end
        else
        
```

```

        Data_Out = 3'bz; // 高阻抗 (High Impedance)
    end
end
endmodule

```



▲ RAM\_8\_3.V 電路的模擬結果



## 12.3 隨機存取記憶體 (RAM) 的應用

隨機存取記憶體 (RAM) 為電路中需要大量儲存資料的地方，而且我們可以將資料寫入到隨機存取記憶體中或者是從隨機存取記憶體中讀出資料。隨機存取記憶體也可以用來模擬：堆疊 (Stack) 的運作、佇列 (Queue) 的運作、串列 (Linked List) 的運作…等等的資料結構的表示方法。在底下的章節中將為各位介紹：使用隨機存取記憶體來模擬：堆疊 (Stack) 的運作、以及佇列 (Queue) 的運作這二種資料結構的表示方法。

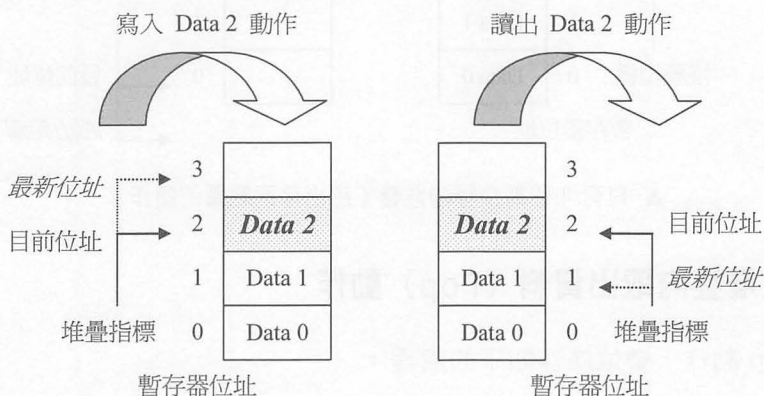
## 12.3.1 堆疊 (Stack) 的運作：使用隨機存取記憶體來模擬

堆疊 (Stack) 為一種資料結構的表示方法，它所代表的特性為：先進後出 (First In Last Out, FILO)，也就是先寫進到堆疊裡的資料最後才會被讀出來。堆疊電路常被用來記錄副程式 (Sub-Routine) 或函數 (Function) 的返回位址以及可以復原至上一個狀態的各類應用...等等的用途上。

我們可以使用隨機存取記憶體來模擬堆疊的運作。因為堆疊也必須由許多個暫存器或者是記憶元件所組合而成，這些暫存器的位元寬度 (Bit Width) 決定了此堆疊的寬度，這些暫存器的個數位決定了此堆疊的深度 (Depth)。

堆疊內部需要有一個用於：記錄將資料寫入到堆疊中或者從堆疊中讀出資料的暫存器，我們稱之為「堆疊指標暫存器」(Stack Pointer)。

堆疊的運作主要分為：置入資料到堆疊中 (Push)，以及從堆疊內取出資料 (Pop)。如圖「具有 4 個暫存器的堆疊 (正常運作)」所示。



▲ 具有 4 個暫存器的堆疊 (正常運作)

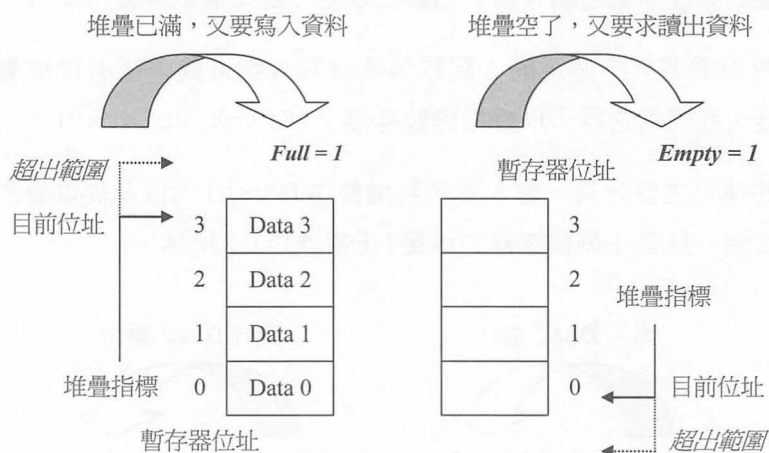
### (一) 置入資料到堆疊中 (Push) 動作

此 Push 動作，會依序作如下的處理：

檢查目前欲寫入資料的位址(堆疊指標暫存器)是不是在最大的有效位址範圍之內？

如果在範圍之內，則將資料寫入到堆疊指標暫存器所指的位址，而且讓堆疊指標暫存器加一(下一個欲寫入資料的位址)。

如果超過最大的有效位址範圍，則發出  $Full = 1$  (堆疊溢滿, Stack Full) 這個旗標訊號。如圖「具有 4 個暫存器的堆疊(超出存取範圍的運作)」的左半部圖所示。



▲ 具有 4 個暫存器的堆疊 (超出存取範圍的運作)

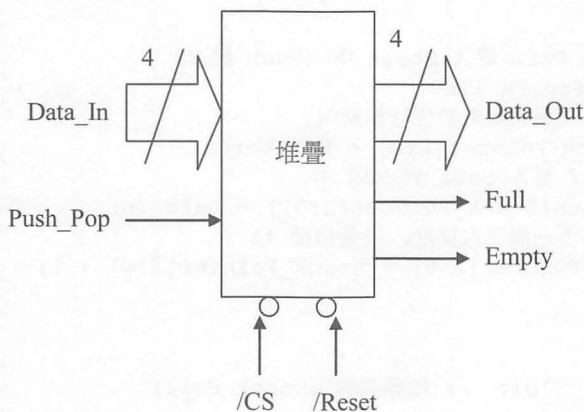
## (二) 從堆疊內取出資料 (Pop) 動作

此 Pop 動作，會依序作如下的處理：

檢查目前欲讀出資料的位址(堆疊指標暫存器)是不是在最小的有效位址範圍之內？

如果在範圍之內，則依據堆疊指標暫存器所指的位址從堆疊內取出資料，而且讓堆疊指標暫存器減一(下一個讀出資料的位址)。如果不在最小的有效位

址範圍，則發出  $\text{Empty}=1$  (堆疊空乏，Stack Empty) 這個旗標訊號。如圖「具有 4 個暫存器的堆疊 (超出存取範圍的運作)」的右半部圖所示。



▲堆疊 (Stack) 電路的邏輯符號

【StackRAM.V：堆疊 (Stack) 的運作，使用隨機存取記憶體來模擬】

```
// StackRAM.V：堆疊 (Stack) 的運作，使用隨機存取記憶體來模擬

module    Stack_By_RAM (Reset, CS, Push_Pop, Data_In, Data_Out,
                        Full, Empty);
parameter    Max_Words = 4;
input       Reset, CS, Push_Pop;
input  [3:0] Data_In;
output [3:0] Data_Out;
reg  [3:0] Data_Out;
output    Full, Empty;
reg      Full, Empty;

reg  [3:0] RAM_Data[Max_Words-1:0];
reg  [2:0] Stack_Pointer;

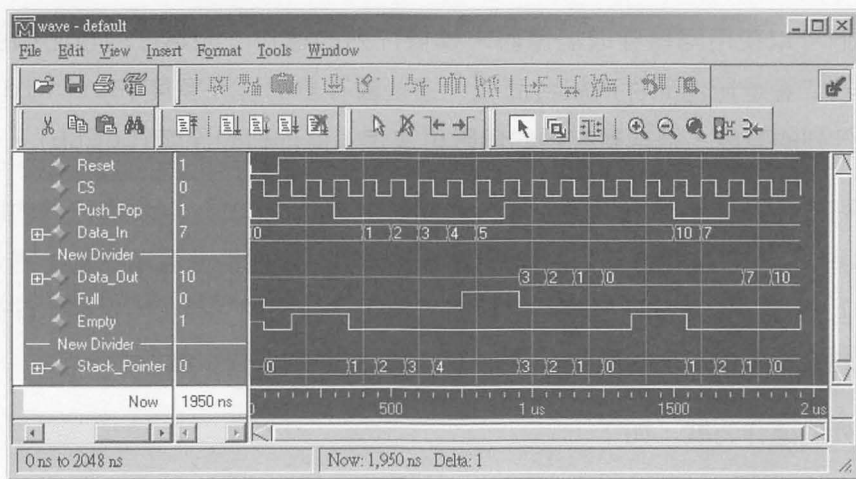
always @(negedge CS)
begin // 負緣觸發：選擇到此晶片
    // 預設每一個 output 的初始值，這樣就不用在底下的每一個 if 中加
    // 預設為沒有發生：
    // 堆疊溢滿 (Stack Full) 及 堆疊空乏 (Stack Empty) 的情形
```

```

{Full, Empty} = 2'b0;
if (Reset == 1'b0) // 電路重置
begin
    Stack_Pointer[2:0] = 0; // 設定：堆疊指標的位址
end
else
begin // 將 Data 置入 Stack 中 (Push 動作)
    if (Push_Pop == 1'b0)
    begin // 在最大有效的位址範圍內
        if (Stack_Pointer[2:0] < Max_Words)
        begin // 寫入 Data 到 RAM 中
            RAM_Data[Stack_Pointer[2:0]] = Data_In;
            // 計算下一個寫入位址，堆疊指標 +1
            Stack_Pointer[2:0] = Stack_Pointer[2:0] + 1;
        end
        else
        begin
            Full = 1'b1; // 堆疊溢滿 (Stack Full)
        end
    end
    else
    begin // 由 Stack 中取出 Data (Pop 動作)
        if (Push_Pop == 1'b1)
        begin // 在最小有效的位址範圍內
            if (Stack_Pointer[2:0] > 0)
            begin // 計算取出資料的位址，堆疊指標 -1
                Stack_Pointer = Stack_Pointer - 1;
                // 從 RAM 中讀出 Data
                Data_Out = RAM_Data[Stack_Pointer[2:0]];
            end
            else
            begin
                Empty = 1'b1; // 堆疊空乏 (Stack Empty)
            end
        end
        else
        begin // Push_Pop 不為 0 (Push 動作) 或 1 (Pop 動作)
            Data_Out = 4'bz; // 高阻抗 (High Impedance)
        end
    end
end
end
end
endmodule

```





▲ StackRAM.V 電路的模擬結果

### 12.3.2 佇列 (Queue) 的運作：使用隨機存取記憶體來模擬

佇列 (Queue) 為一種資料結構的表示方法，它所代表的特性為：先進先出 (First In First Out, FIFO)，也就是先寫進到堆疊裡的資料也是最早被讀出來的。

佇列電路常被用來當作資料的緩衝器 (Data Buffer)，以及事件的排程 (Event Schedule) …等等的用途上。當作資料緩衝器使用時，可使資料之間的傳輸速度維持在一個較平穩的狀態；例如：網路介面卡上的緩衝器、印表機的緩衝器、CPU 內部的指令緩衝器…等等。當作事件的排程使用時，可以將事件發生的順序依序地記錄下來，再依照欲處理的順序依序地處理。

佇列內部有二個位址暫存器，一個用於記錄從佇列中讀出資料的指標 (資料讀出指標, Tail Pointer)，另一個則是用於記錄將資料寫入到佇列中的指標 (資料寫入指標, Head Pointer)。資料讀出指標所指的位址為下次的讀出操作時，該被讀出資料的位址。資料寫入指標所指的位址為下次的寫入操作時，該被寫入資料的位址。

佇列的運作主要分為：置入資料到佇列中 (Insert)，以及從佇列內取出資料 (Delete)。並且配合佇列的狀態旗標來檢知：佇列是否溢滿不能再寫入資料的 Full 旗標，以及佇列是否空乏沒有資料可供讀出的 Empty 旗標。佇列的詳細運作方式請看接下來的數個「具有 10 個暫存器的佇列」圖所示。

### (一) 佇列的初始狀態

Tail 指標      Head 指標

目前位址      目前位址

Full = 0

Empty = 1

暫存器位址      0      1      2      3

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

9      4

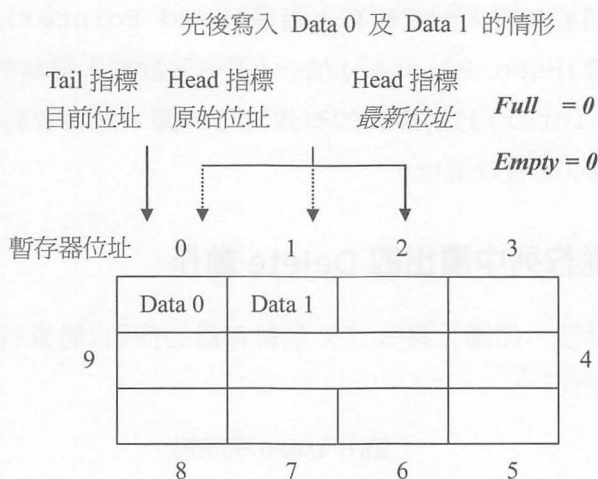
8      7      6      5

一開始佇列的資料讀出指標 (Tail) 與資料寫入指標 (Head) 的目前位址均指向暫存器位址 0，狀態旗標：佇列溢滿 Full 旗標設為 0 (代表佇列仍可寫入資料)，以及佇列空乏 Empty 旗標設為 1 (代表佇列內沒有資料可供讀出)。

如果在此時要求從佇列中強制讀出一筆資料，則因為佇列是空的，所以無法讀出資料來；我們可以藉由判斷狀態旗標：佇列是否空乏沒有資料可供讀出的 Empty 旗標來達成此項判斷動作。

## (二) 將資料寫入到佇列中的 Insert 動作

佇列的初始狀態，如圖「具有 10 個暫存器的佇列 (將資料寫入到佇列中的 Insert 動作)」所示。



▲ 具有 10 個暫存器的佇列 (將資料寫入到佇列中的 Insert 動作)

如果佇列目前是空乏的 (Empty 旗標為 1，沒有資料可供讀出)

在此狀態下佇列的 Insert 動作，會依序作如下的處理：

將資料寫入到資料寫入指標 (Head Pointer) 所指的位址。同時讓資料寫入指標 (Head Pointer) 加一 (下一個欲寫入資料的位址)：如果寫入指標 (Head Pointer) 超過最大的有效位址範圍，將資料寫入指標 (Head Pointer) 設為最小的有效位址。最後將佇列空乏 Empty 旗標設為 0 (代表佇列內有資料可供讀出)。

如果佇列目前不是空乏的（Empty 旗標為 0，有資料可供讀出）

在此狀態下佇列的 Insert 動作，會依序作如下的處理：

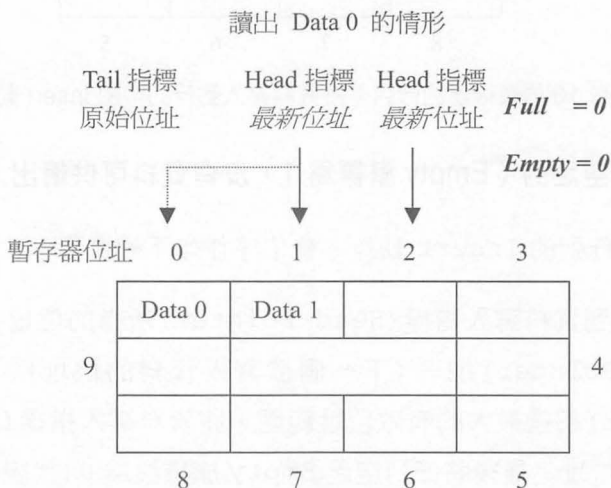
首先比較資料寫入指標 (Head Pointer) 與資料讀出指標 (Tail Pointer) 是否相同？

如果相同，將佇列溢滿 Full 旗標設為 1（代表佇列已滿，無法再將資料寫入到佇列中）。

如果不同，將資料寫入到資料寫入指標 (Head Pointer) 所指的位址。同時讓資料寫入指標 (Head Pointer) 加一（下一個欲寫入資料的位址）：如果寫入指標 (Head Pointer) 超過最大的有效位址範圍，則將資料寫入指標 (Head Pointer) 設為最小的有效位址。

### （三）將資料從佇列中讀出的 Delete 動作

佇列的初始狀態，如圖「具有 10 個暫存器的佇列（將資料從佇列中讀出的 Delete 動作）」所示。



▲ 具有 10 個暫存器的佇列（將資料從佇列中讀出的 Delete 動作）

如果佇列目前是空乏的（Empty 旗標為 1，沒有資料可供讀出）

在此狀態下佇列的 Delete 動作，將無法從佇列中讀出資料。

如果佇列目前不是空乏的（Empty 旗標為 0，有資料可供讀出）

在此狀態下佇列的 Delete 動作，會依序作如下的處理：

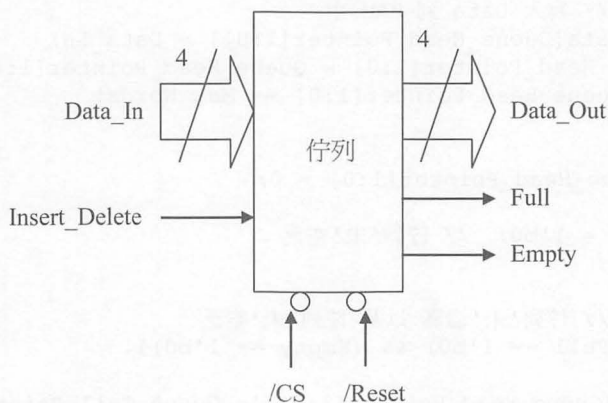
依資料讀出指標 (Tail Pointer) 所指向佇列中的位址讀出資料。同時讓資料讀出指標 (Tail Pointer) 加一（下一個欲讀出資料的位址），如果資料讀出指標 (Tail Pointer) 超過最大的有效位址範圍，則將資料讀出指標 (Tail Pointer) 設為最小的有效位址。

將佇列空乏溢滿 Full 旗標設為 0（代表佇列未滿，仍有空間供資料寫入到佇列中）。

再比較資料寫入指標 (Head Pointer) 與資料讀出指標 (Tail Pointer) 是否相同？

如果不同，將佇列空乏 Empty 旗標設為 0（代表佇列內有資料可供讀出）。

如果相同，將佇列空乏 Empty 旗標設為 1（代表佇列內沒有資料可供讀出）。



▲ 佇列 (Queue) 電路的邏輯符號

### 【QueueRAM.V：佇列 (Queue) 的運作，使用隨機存取記憶體來模擬】

// QueueRAM.V：佇列 (Queue) 的運作，使用隨機存取記憶體來模擬

```

module Queue_By_RAM (Reset, CS, Insert_Delete, Data_In,
    Data_Out, Full, Empty);
    parameter Max_Words = 4;
    input      Reset, CS, Insert_Delete;
    input  [3:0] Data_In;
    output  [3:0] Data_Out;
    reg     [3:0] Data_Out;
    output      Full, Empty;
    reg         Full, Empty;

    reg  [3:0] RAM_Data[Max_Words-1:0];
    reg  [1:0] Queue_Head_Pointer, Queue_Tail_Pointer;

    always @(negedge CS)
    begin // 負緣觸發：重置 或 選擇到此晶片
        if (Reset == 1'b0) // 電路重置
            begin
                Queue_Head_Pointer[1:0] = 0; // 設定：佇列指標開頭的位址
                Queue_Tail_Pointer[1:0] = 0; // 設定：佇列指標結尾的位址
                Full = 1'b0; // 佇列'未'溢滿
                Empty = 1'b1; // 佇列 空乏
            end
        else
            begin // 將 Data 置入 Queue 中 (Insert 動作)
                if (Insert_Delete == 1'b0)
                    begin // 佇列'未'溢滿 以及 佇列空乏
                        if ((Full == 1'b0) && (Empty == 1'b1))
                            begin // 寫入 Data 到 RAM 中
                                RAM_Data[Queue_Head_Pointer[1:0]] = Data_In;
                                Queue_Head_Pointer[1:0] = Queue_Head_Pointer[1:0] + 1;
                                if (Queue_Head_Pointer[1:0] == Max_Words)
                                    begin
                                        Queue_Head_Pointer[1:0] = 0;
                                    end
                                Empty = 1'b0; // 佇列'未'空乏
                            end
                        else
                            begin // 佇列'未'溢滿 以及 佇列'未'空乏
                                if ((Full == 1'b0) && (Empty == 1'b0))
                                    begin
                                        if (Queue_Head_Pointer[1:0] != Queue_Tail_Pointer[1:0])
                                            begin // 寫入 Data 到 RAM 中

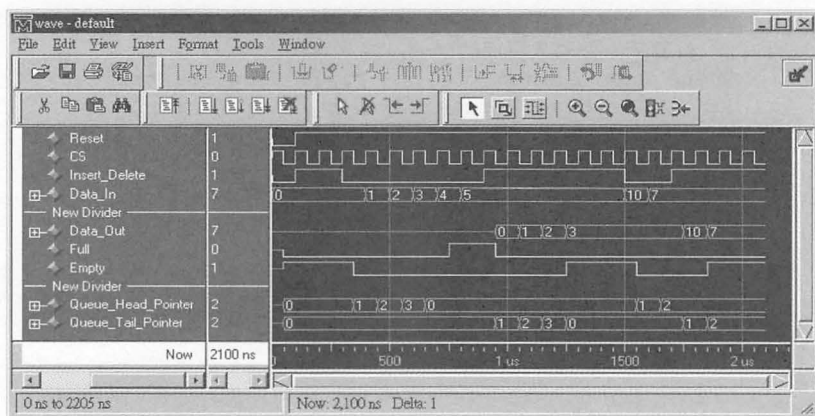
```



```

    RAM_Data[Queue_Head_Pointer[1:0]] = Data_In;
    // 計算下一個寫入資料的位址, 寫入資料指標 +1
    Queue_Head_Pointer[1:0] = Queue_Head_Pointer[1:0] + 1;
    if (Queue_Head_Pointer[1:0] == Max_Words)
    begin
        Queue_Head_Pointer[1:0] = 0;
    end
end
else
begin
    Full = 1'b1; // 佇列溢滿 (Queue Full)
end
end
end
end
else
begin // 從 Queue 中取出 Data (Delete 動作)
    if (Insert_Delete == 1'b1)
    begin // 佇列'未'溢滿 以及 佇列'未'空乏
        if (Empty == 1'b0) // 佇列'未'空乏
        begin // 從 RAM 中 讀出 Data
            Data_Out = RAM_Data[Queue_Tail_Pointer[1:0]];
            // 計算下一個資料的讀出位址, 讀出資料指標 +1
            Queue_Tail_Pointer[1:0] = Queue_Tail_Pointer[1:0] + 1;
            if (Queue_Tail_Pointer[1:0] == Max_Words)
            begin
                Queue_Tail_Pointer[1:0] = 0;
            end
            Full = 1'b0; // 佇列'未'溢滿
            if (Queue_Tail_Pointer[1:0] != Queue_Head_Pointer[1:0])
            begin
                Empty = 1'b0; // 佇列'未'空乏
            end
        else
        begin
            Empty = 1'b1; // 佇列空乏 (Queue Empty)
        end
    end
end
end
else
begin // Insert_Delete 不為 0 (Insert 動作) 或 1 (Delete 動作)
    Data_Out = 4'bz; // 高阻抗 (High Impedance)
end
end
end
end
endmodule

```



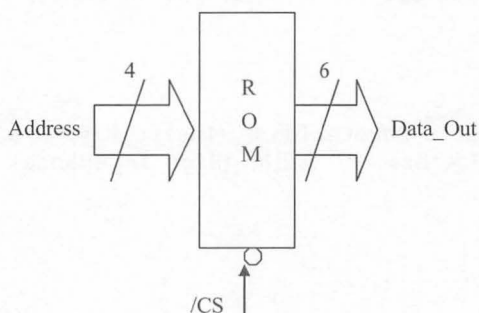
▲ QueueRAM.V 合成後，電路的模擬結果



## 12.4 唯讀記憶體 (Read Only Memory, ROM)

唯讀記憶體用於儲存固定不會再更動的值，它由一些基本的電晶體 (Transistor) 與位址解碼器 (Address Decoder) 與讀取動作的邏輯閘電路所組成的。而將其內的電晶體部份連到  $V_{dd}$  與部份連到  $V_{ss}$ ，分別代表固定為 1 以及固定為 0。

通常唯讀記憶體都有以下的控制訊號：CS (晶片選擇，Chip Select)、Address (位址線) 以及 Data\_Out (資料輸出線)。



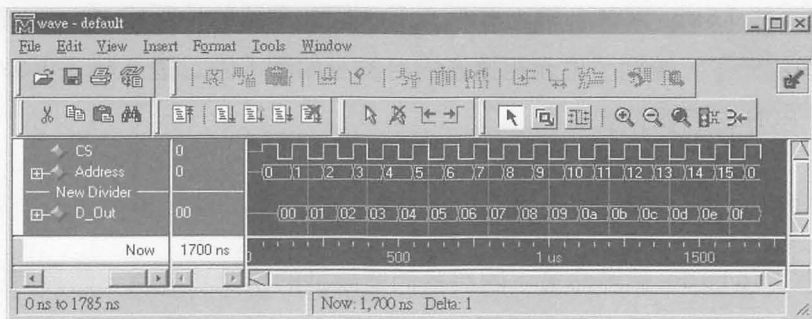
▲ 具有 16 個字組，每個字組有 6 個位元的唯讀記憶體的邏輯符號

## 【ROMW16B6.V：具有 16 個字組，每個字組有 6 個位元的唯讀記憶體】

```
// ROMW16B6.V：具有 16 個字組，每個字組有 6 個位元的唯讀記憶體
module ROM (CS, Address, D_Out);
input      CS;
input      [3:0] Address;
output     [5:0] D_Out;
reg        [5:0] D_Out;
// 使用 16 個 6 Bits 的 Register，來模擬 ROM
reg        [5:0] ROM_Data[15:0];

always @(negedge CS) // 選擇到此晶片
begin

    ROM_Data [ 0] = 6'd0; // 讓位址為 0 的內容固定為 0
    ROM_Data [ 1] = 6'd1; // 讓位址為 1 的內容固定為 1
    ROM_Data [ 2] = 6'd2; // 讓位址為 2 的內容固定為 2
    ROM_Data [ 3] = 6'd3; // 讓位址為 3 的內容固定為 3
    ROM_Data [ 4] = 6'd4; // 讓位址為 4 的內容固定為 4
    ROM_Data [ 5] = 6'd5; // 讓位址為 5 的內容固定為 5
    ROM_Data [ 6] = 6'd6; // 讓位址為 6 的內容固定為 6
    ROM_Data [ 7] = 6'd7; // 讓位址為 7 的內容固定為 7
    ROM_Data [ 8] = 6'd8; // 讓位址為 8 的內容固定為 8
    ROM_Data [ 9] = 6'd9; // 讓位址為 9 的內容固定為 9
    ROM_Data[10] = 6'd10; // 讓位址為 10 的內容固定為 10
    ROM_Data[11] = 6'd11; // 讓位址為 11 的內容固定為 11
    ROM_Data[12] = 6'd12; // 讓位址為 12 的內容固定為 12
    ROM_Data[13] = 6'd13; // 讓位址為 13 的內容固定為 13
    ROM_Data[14] = 6'd14; // 讓位址為 14 的內容固定為 14
    ROM_Data[15] = 6'd15; // 讓位址為 15 的內容固定為 15
    D_Out = ROM_Data[Address]; // 由 ROM_Data 內取出資料
end
endmodule
```



▲ ROMW16B6.V 合成後，電路的模擬結果



## 12.5 唯讀記憶體 (ROM) 的擴充

唯讀記憶體擴充的方式也是和隨機存取記憶體擴充的方式一樣，分成：位元組 (Bits) 的擴充以及字組 (Words) 的擴充這兩種方式。

有一個限制條件是：必須要是同樣位元組 (Bits) 大小以及字組 (Words) 大小，而且具有相同控制訊號功能的 ROM 才可以組成唯讀記憶體的擴充電路。

### 12.5.1 唯讀記憶體之位元組 (Bits) 的擴充方式

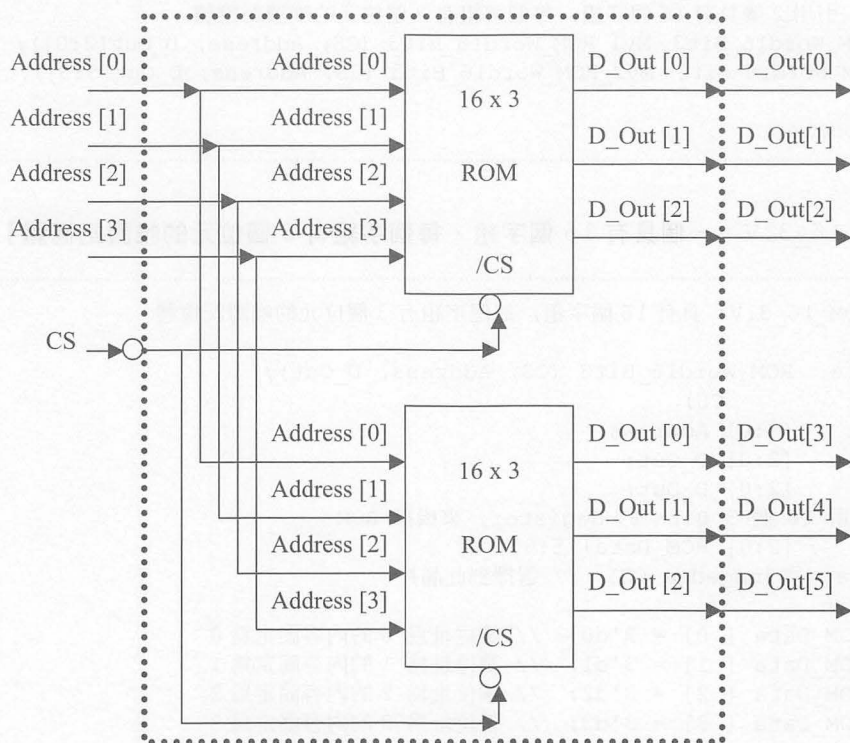
例如：我們需要設計出一個具有 16 個字組，每個字組有 6 個位元的唯讀記憶體模組電路 (16x6 ROM)，則必須利用兩個具有 16 個字組，每個字組有 3 個位元的唯讀記憶體模組 (16x3 ROM)，先將它們所有的晶片選擇線 (Chip Select) 與位址線 (Address) 全部並聯起來當作是 16x6 ROM 的晶片選擇線 (Chip Select) 及位址線 (Address)。再將其中一個 16x3 ROM 的資料輸出線 (Data Out) 接到 16x6 ROM 資料輸出線 (Data Out) 的位元 2 到位元 0 (Data\_Out[2:0])，另一個 16x3 ROM 的資料輸出線 (Data Out) 接到 16x6 ROM 資料輸出線 (Data Out) 的位元 5 到位元 3 (Data\_Out[5:3])。

利用將這兩個 16x3 ROM 的晶片選擇線 (Chip Select) 並聯起來的目的在於：這兩個 16x3 ROM 均同時被選擇到或者是沒有被選擇到，若同時被選擇到，則這兩個 16x3 ROM 各輸出 3 個位元；因此電路的位元組 (Bits) 數目就結合為 6 個位元了，達到了位元組擴充的目的。如圖「由二個 16x3 ROM 組成一個 16x6 ROM 的邏輯符號」。

**範例** ROM\_16\_6.V：位元組 (Bits) 的擴充範例

利用 ``include` 編譯命令：因為我們要設計的模組功能為一個具有 16 個字組，每個字組有 6 個位元的唯讀記憶體的 Verilog 電路描述檔 (ROM\_16\_

6.V)。因此我們可以引用已經設計好的一個具有 16 個字組，每個字組有 3 個位元的唯讀記憶體的 Verilog 電路描述檔 (ROM\_16\_3.V) 來一起編譯合成。



▲ ROM\_16\_6.V：由二個 16x3 ROM 組成一個 16x6 ROM 的邏輯符號

【ROM\_16\_6.V：一個具有 16 個字組，每個字組有 6 個位元的唯讀記憶體】

```
// ROM_16_6.V
// 運用 2 個具有 16 個字組，每個字組有 3 個位元的唯讀記憶體
// 來設計出 1 個具有 16 個字組，每個字組有 6 個位元的唯讀記憶體
// 引用 ROM_16_3.V
// 一個具有 16 個字組，每個字組有 3 個位元的唯讀記憶體
`include "ROM_16_3.V"

// 具有 16 個字組，每個字組有 6 個位元的唯讀記憶體
module ROM_Word16_Bit6 (CS, Address, D_Out);
```

```
input          CS;
output [5:0] D_Out;
input  [3:0] Address;
wire   [5:0] D_Out;

// 引用 2 個具有 16 個字組，每個字組有 3 個位元的唯讀記憶體
ROM_Word16_Bit3 My1_ROM_Word16_Bit3 (CS, Address, D_Out[2:0]);
ROM_Word16_Bit3 My2_ROM_Word16_Bit3 (CS, Address, D_Out[5:3]);

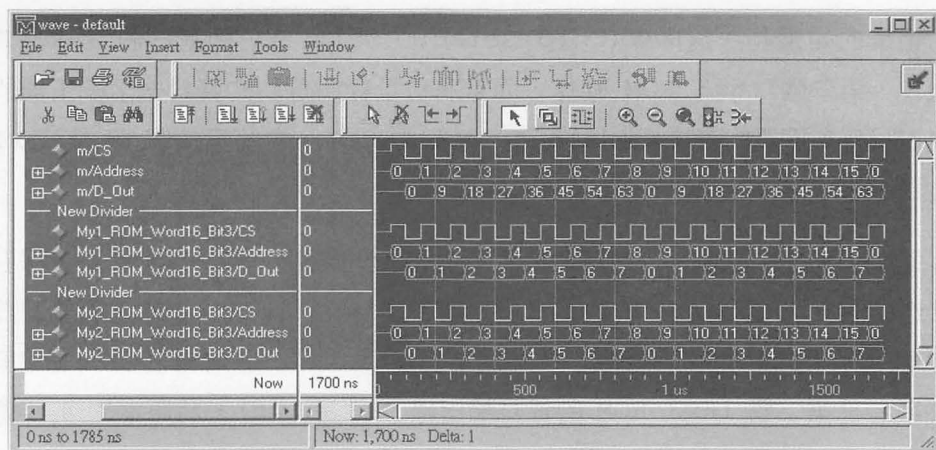
Endmodule
```

【ROM\_16\_3.V：一個具有 16 個字組，每個字組有 3 個位元的唯讀記憶體】

```
// ROM_16_3.V: 具有 16 個字組，每個字組有 3 個位元的唯讀記憶體

module ROM_Word16_Bit3 (CS, Address, D_Out);
input          CS;
input  [3:0] Address;
output [2:0] D_Out;
reg [2:0] D_Out;
// 使用 16 個 3 Bits 的 Register，來模擬 ROM
reg [2:0] ROM_Data[15:0];
always @(negedge CS) // 選擇到此晶片
begin
    ROM_Data [ 0] = 3'd0; // 讓位址為 0 的內容固定為 0
    ROM_Data [ 1] = 3'd1; // 讓位址為 1 的內容固定為 1
    ROM_Data [ 2] = 3'd2; // 讓位址為 2 的內容固定為 2
    ROM_Data [ 3] = 3'd3; // 讓位址為 3 的內容固定為 3
    ROM_Data [ 4] = 3'd4; // 讓位址為 4 的內容固定為 4
    ROM_Data [ 5] = 3'd5; // 讓位址為 5 的內容固定為 5
    ROM_Data [ 6] = 3'd6; // 讓位址為 6 的內容固定為 6
    ROM_Data [ 7] = 3'd7; // 讓位址為 7 的內容固定為 7
    ROM_Data [ 8] = 3'd0; // 讓位址為 8 的內容固定為 0
    ROM_Data [ 9] = 3'd1; // 讓位址為 9 的內容固定為 1
    ROM_Data [10] = 3'd2; // 讓位址為 10 的內容固定為 2
    ROM_Data [11] = 3'd3; // 讓位址為 11 的內容固定為 3
    ROM_Data [12] = 3'd4; // 讓位址為 12 的內容固定為 4
    ROM_Data [13] = 3'd5; // 讓位址為 13 的內容固定為 5
    ROM_Data [14] = 3'd6; // 讓位址為 14 的內容固定為 6
    ROM_Data [15] = 3'd7; // 讓位址為 15 的內容固定為 7
    D_Out = ROM_Data[Address]; // 由 ROM_Data 內取出資料
end
endmodule
```





▲ ROM\_16\_6.V 電路的模擬結果

## 12.5.2 唯讀記憶體之字組 (Words) 的擴充方式

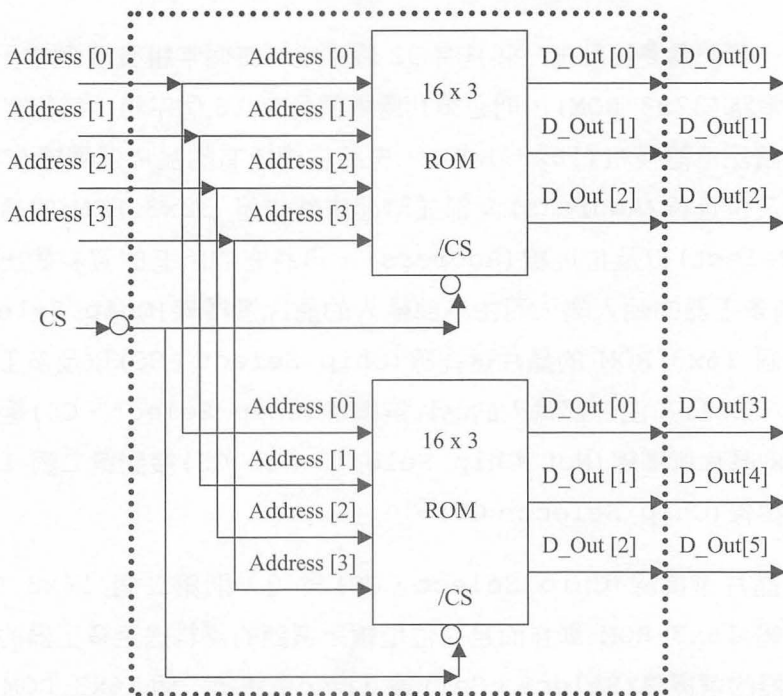
例如：我們需要設計出一個具有 32 個字組，每個字組有 3 個位元的唯讀記憶體模組電路 ( $32 \times 3$  ROM)，則必須利用兩個具有 16 個字組，每個字組有 3 個位元的唯讀記憶體模組 ( $16 \times 3$  ROM)，先將它們所有的晶片選擇線 (Chip Select) 以及位址線 (Address) 全部並聯起來當作是  $32 \times 3$  ROM 的晶片選擇線 (Chip Select) 以及位址線 (Address)。再將它們所有的資料輸出線 (Data Out) 接到多工器的輸入端，而由外部輸入的晶片選擇線 (Chip Select, CS) 接到第一個  $16 \times 3$  ROM 的晶片選擇線 (Chip Select, CS) 以及多工器選擇線 (Select, Sel)，由外部輸入的晶片選擇線 (Chip Select, CS) 經過一個反向器的反向晶片選擇線 (Not Chip Select, Not\_CS) 接到第二個  $16 \times 3$  ROM 的晶片選擇線 (Chip Select, CS)。

如果晶片選擇線 (Chip Select, CS) 為 0，則第二個  $16 \times 3$  ROM 不動作，第一個  $16 \times 3$  ROM 動作而且將位址線所選到的資料送至多工器的輸入端，同時多工器的選擇線 (Select, Sel) 為 0 時，會將第一個  $16 \times 3$  ROM 的輸出資料送到整個  $32 \times 3$  ROM 的資料輸出線 (Data Out)。



相反如果晶片選擇線 (Chip Select, CS) 為 1，則第一個 16x3 ROM 不動作，第二個 16x3 ROM 動作，而且將位址線所選到的資料送至多工器的輸入端，同時多工器的選擇線 (Select, Sel) 為 1 時，會將第二個 16x3 ROM 的輸出資料送到整個 32x3 ROM 的資料輸出線 (Data Out)。

這樣一來，這兩個 16x3 ROM 同一時間只有一個 16x3 ROM 會被選擇到，而且也會因為由 32x3 ROM 的晶片選擇線 (Chip Select, CS) 連到多工器的選擇線 (Select, Sel) 來決定資料輸出 (Data Out) 是來自於第一個 16x3 ROM 或者是第二個 16x3 ROM 的資料輸出線 (Data Out)。將原本的 4 條位址線再加上晶片選擇線 (Chip Select, CS) 合成 5 條線 (2 的 5 次方等於 32) 來控制電路的字組 (Words) 數目，就合成了 32x3 ROM，達到字組擴充的目的。如圖「由二個 16x3 ROM 組成一個 32x3 ROM 的邏輯符號」。



▲ ROM\_32\_3.V：由二個 16x3 ROM 組成一個 32x3 ROM 的邏輯符號

**範例** ROM\_32\_3.V：位元組(Bits)的擴充範例

利用`include 編譯命令：因為我們要設計的模組功能為一個具有 32 個字組，每個字組有 3 個位元的唯讀記憶體的 Verilog 電路描述檔(ROM\_32\_3.V)，因此我們可以引用已經設計好的一個具有 16 個字組，每個字組有 3 個位元的唯讀記憶體的 Verilog 電路描述檔(R\_16\_3\_1.V 及 R\_16\_3\_2.V，其 ROM 內的資料不同而已)來一起編譯合成。

**【ROM\_32\_3.V：一個具有 16 個字組，每個字組有 6 個位元的唯讀記憶體】**

```
// ROM_32_3.V
// 運用 2 個具有 16 個字組，每個字組有 3 個位元的唯讀記憶體
// 來設計出 1 個具有 32 個字組，每個字組有 3 個位元的唯讀記憶體

// 引用 R_16_3_1.V 及 R_16_3_2.V
// 均為 1 個具有 16 個字組，每個字組有 3 個位元的唯讀記憶體
// 且所有的訊號線皆相同，唯獨其 ROM 內的資料不同而已
`include "R_16_3_1.V" // ROM 內的資料 和 R_16_3_2.V 不同
`include "R_16_3_2.V" // ROM 內的資料 和 R_16_3_1.V 不同
// 具有 32 個字組，每個字組有 3 個位元的唯讀記憶體
module ROM_Word32_Bit3 (CS, Address, D_Out);
input      CS;
input      [3:0] Address;
output     [2:0] D_Out;
reg        [2:0] D_Out;
wire       Not_CS;
wire       [2:0] D_Out_1;
wire       [2:0] D_Out_2;

// 作 Invert Chip Select 訊號
assign Not_CS = !CS;

// 引用 2 個具有 16 個字組，每個字組有 3 個位元的唯讀記憶體
ROM_Word16_Bit3_1 My1_ROM_Word16_Bit3 (CS, Address, D_Out_1);
ROM_Word16_Bit3_2 My2_ROM_Word16_Bit3 (Not_CS, Address, D_Out_2);
always @(CS or D_Out_1 or D_Out_2)
begin
    if (CS == 1'b0)
    begin
        D_Out[2:0] = D_Out_1[2:0];
    end
end
```

```

        if (CS == 1'b1)
        begin
            D_Out[2:0] = D_Out_2[2:0];
        end
    end
endmodule

```

【R\_16\_3\_1.V：具有 16 個字組，每個字組有 3 個位元的唯讀記憶體，ROM 內的資料和 R\_16\_3\_2.V 不同】

```

// R_16_3_1.V：具有 16 個字組，每個字組有 3 個位元的唯讀記憶體
//          ROM 內的資料 和 R_16_3_2.V 不同

module    ROM_Word16_Bit3_1 (CS, Address, D_Out);
input     CS;
input     [3:0] Address;
output    [2:0] D_Out;
reg       [2:0] D_Out;
// 使用 16 個 3 Bits 的 Register，來模擬 ROM
reg       [2:0] ROM_Data[15:0];
    always @(negedge CS) // 選擇到此晶片
    begin
        ROM_Data [ 0] = 3'd0; // 讓位址為 0 的內容固定為 0
        ROM_Data [ 1] = 3'd1; // 讓位址為 1 的內容固定為 1
        ROM_Data [ 2] = 3'd2; // 讓位址為 2 的內容固定為 2
        ROM_Data [ 3] = 3'd3; // 讓位址為 3 的內容固定為 3
        ROM_Data [ 4] = 3'd4; // 讓位址為 4 的內容固定為 4
        ROM_Data [ 5] = 3'd5; // 讓位址為 5 的內容固定為 5
        ROM_Data [ 6] = 3'd6; // 讓位址為 6 的內容固定為 6
        ROM_Data [ 7] = 3'd7; // 讓位址為 7 的內容固定為 7

        ROM_Data [ 8] = 3'd0; // 讓位址為 8 的內容固定為 0
        ROM_Data [ 9] = 3'd1; // 讓位址為 9 的內容固定為 1
        ROM_Data [10] = 3'd2; // 讓位址為 10 的內容固定為 2
        ROM_Data [11] = 3'd3; // 讓位址為 11 的內容固定為 3
        ROM_Data [12] = 3'd4; // 讓位址為 12 的內容固定為 4
        ROM_Data [13] = 3'd5; // 讓位址為 13 的內容固定為 5
        ROM_Data [14] = 3'd6; // 讓位址為 14 的內容固定為 6
        ROM_Data [15] = 3'd7; // 讓位址為 15 的內容固定為 7
        D_Out = ROM_Data[Address]; // 由 ROM_Data 內取出資料
    end
endmodule

```

【R\_16\_3\_2.V：具有 16 個字組，每個字組有 3 個位元的唯讀記憶體，ROM 內的資料和 R\_16\_3\_1.V 不同】

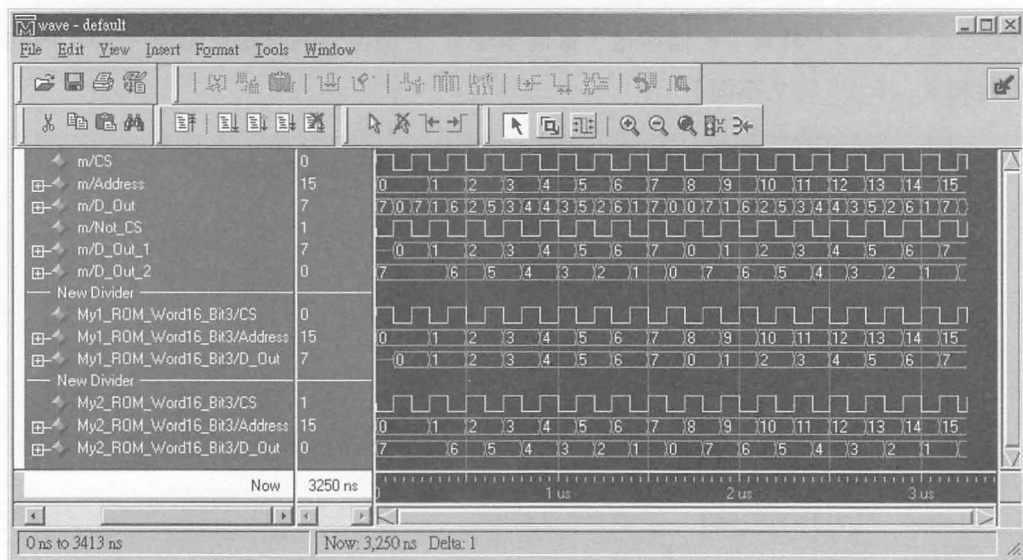
```
// R_16_3_2.V: 具有 16 個字組，每個字組有 3 個位元的唯讀記憶體
// ROM 內的資料 和 R_16_3_1.V 不同
```

```
module ROM_Word16_Bit3_2 (CS, Address, D_Out);
input CS;
input [3:0] Address;
output [2:0] D_Out;

reg [2:0] D_Out;
// 使用 16 個 3 Bits 的 Register, 來模擬 ROM
reg [2:0] ROM_Data[15:0];

always @(negedge CS) // 選擇到此晶片
begin
    ROM_Data [ 0] = 3'd7; // 讓位址為 0 的內容固定為 7
    ROM_Data [ 1] = 3'd6; // 讓位址為 1 的內容固定為 6
    ROM_Data [ 2] = 3'd5; // 讓位址為 2 的內容固定為 5
    ROM_Data [ 3] = 3'd4; // 讓位址為 3 的內容固定為 4
    ROM_Data [ 4] = 3'd3; // 讓位址為 4 的內容固定為 3
    ROM_Data [ 5] = 3'd2; // 讓位址為 5 的內容固定為 2
    ROM_Data [ 6] = 3'd1; // 讓位址為 6 的內容固定為 1
    ROM_Data [ 7] = 3'd0; // 讓位址為 7 的內容固定為 0

    ROM_Data [ 8] = 3'd7; // 讓位址為 8 的內容固定為 7
    ROM_Data [ 9] = 3'd6; // 讓位址為 9 的內容固定為 6
    ROM_Data [10] = 3'd5; // 讓位址為 10 的內容固定為 5
    ROM_Data [11] = 3'd4; // 讓位址為 11 的內容固定為 4
    ROM_Data [12] = 3'd3; // 讓位址為 12 的內容固定為 3
    ROM_Data [13] = 3'd2; // 讓位址為 13 的內容固定為 2
    ROM_Data [14] = 3'd1; // 讓位址為 14 的內容固定為 1
    ROM_Data [15] = 3'd0; // 讓位址為 15 的內容固定為 0
    D_Out = ROM_Data[Address]; // 由 ROM_Data 內取出資料
end
endmodule
```



▲ ROM\_32\_3.V 合成後，電路的模擬結果



## 12.6 唯讀記憶體 (ROM) 的應用

唯讀記憶體 (ROM) 為電路中需要大量儲存資料的地方，而且這些資料只供讀取而無法寫入。唯讀記憶體在電路中廣泛地被使用到，例如：早期主機板上的 BIOS (Basic Input/Output System)、網路卡上的 BIOS、印表機內建的字型 ROM... 等等的應用。在底下的章節中將為各位介紹：將 ROM 內存放‘指令’的「指令唯讀記憶體」(Instruction ROM)，以及將 ROM 內存放‘資料’的「資料唯讀記憶體」(Data ROM)。

### 12.6.1 指令唯讀記憶體 (Instruction ROM)

指令唯讀記憶體 (Instruction ROM) 和一般的唯讀記憶體 (ROM)，就其基本的電路架構來說都是一樣的，如果某一顆 ROM 內存放的資料可用來作為某

種特定算術單元的「指令碼」，則稱此顆 ROM 為指令唯讀記憶體 (Instruction ROM)。

## 12.6.2 資料唯讀記憶體 (Data ROM)

資料唯讀記憶體 (Data ROM) 和一般的唯讀記憶體 (ROM)，就其基本的電路架構來說都是一樣的，如果某一顆 ROM 內存放的資料可用來作為某種特定算術單元的「資料碼」，則稱此顆 ROM 為資料唯讀記憶體 (Data ROM)。

## 12.6.3 使用一顆指令 ROM 與一顆資料 ROM 來推動特定功能的 ALU

算術邏輯運算單元 (ALU) 為結合了數位系統中最基本的：加法、減法、AND... 等等的運算，由運算碼 (operator) 來決定您想對輸入的資料作那一種的運算。我們使用一顆指令 ROM 與一顆資料 ROM 來推動特定功能的 ALU，來說明 ROM 在電路中的使用方法。

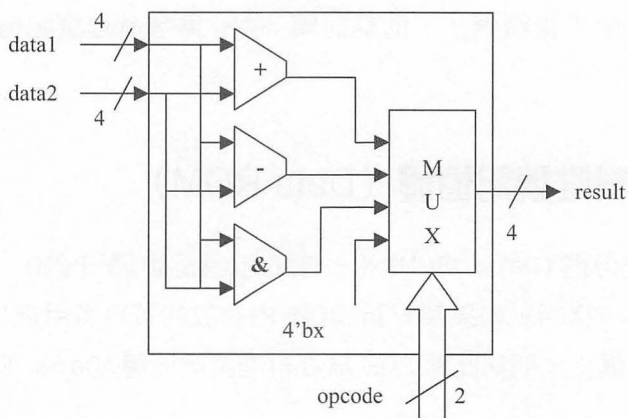
**範例** ROM\_ALU.V：使用一顆指令 ROM 與一顆資料 ROM 來推動特定功能的 ALU

因為我們要設計的模組需要引用一顆指令 ROM (Inst\_ROM.V)、與一顆資料 ROM (Data\_ROM.V) 以及一個簡易的算術邏輯運算單元 (ALU\_2.V)，因此我們利用 ``include` 編譯命令來引用設計好的這些模組。如圖「使用一顆指令 ROM 與一顆資料 ROM 來推動特定功能的 ALU 的邏輯符號」。

**Step1** ALU\_2.V：簡易的算術邏輯運算單元 (ALU)，範例

在 ALU\_2.V 中：運算碼 (operator) 可以是加法、減法、AND... 等其中一種。只要輸入的資料或是運算碼 (operator) 有任何的改變後，就會重新計算。





▲ ALU\_2.V：簡易的算術邏輯運算單元（ALU）的邏輯符號

### 【ALU\_2.V：簡易的算術邏輯運算單元（ALU）】

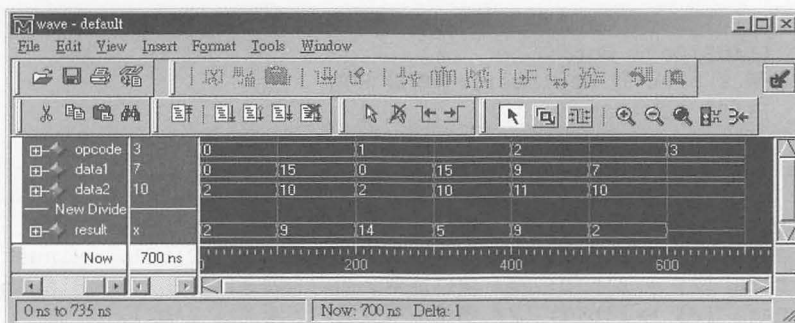
```
// ALU_2.V：簡易的算術邏輯運算單元（ALU），範例

`define Plus      2'd0
`define Minus     2'd1
`define Bin_And   2'd2

module ALU_2 (opcode, data1, data2, result);
input  [1:0] opcode;
input  [3:0] data1, data2;
output [3:0] result;
reg     [3:0] result;
always @(opcode or data1 or data2)
begin
    case (opcode)
        // Arithmetic operations
        `Plus:
        `Minus:
            result = data1 + data2;
            result = data1 - data2;

        // Bitwise operations
        `Bin_And:
            result = data1 & data2;

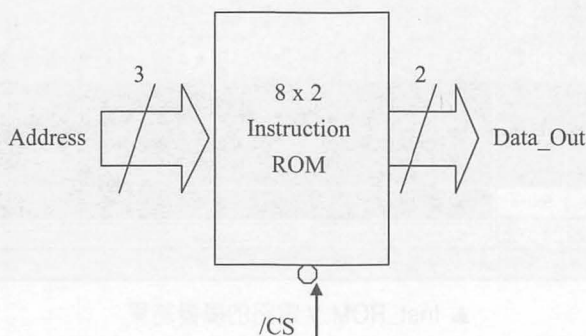
        default:
            result = 4'hx;
    endcase
end
endmodule
```



▲ ALU\_2.V 電路的模擬結果

**Step2** Inst\_ROM.V：存放 8 個字組，每個字組有 2 個位元的'指令'唯讀記憶體

我們將：2'b00、2'b01、2'b10、2'b11、2'b00、2'b01、2'b10 以及 2'b11 等 8 個字組存放在 Inst\_ROM.V 中，作為推動 ALU\_2.V 簡易的算術邏輯運算單元 (ALU) 的指令碼來源。



▲ Inst\_ROM.V：存放 8 個字組，每個字組有 2 個位元的'指令'唯讀記憶體的邏輯符號

【Inst\_ROM.V：存放 8 個字組，每個字組有 2 個位元的'指令'唯讀記憶體】

```

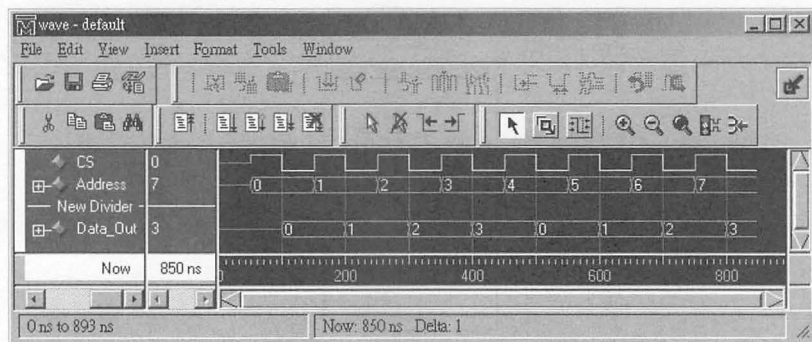
// Inst_ROM.V：存放'指令'的唯讀記憶體
// 具有 8 個字組，每個字組有 2 個位元的唯讀記憶體
module Inst_ROM (CS, Address, Data_Out);
input          CS;
input  [2:0] Address;
  
```

```

output [1:0] Data_Out;
reg [1:0] Data_Out;
// 使用 4 個 8 Bits 的 Register, 來模擬 ROM
reg [1:0] ROM_Data[7:0];

always @(negedge CS) // 選擇到此晶片
begin
    ROM_Data [0] = 2'b00; // 讓位址為 0 的內容固定為 2'b00, `Plus
    ROM_Data [1] = 2'b01; // 讓位址為 1 的內容固定為 2'b01, `Minus
    ROM_Data [2] = 2'b10; // 讓位址為 2 的內容固定為 2'b10, `Bin_And
    ROM_Data [3] = 2'b11; // 讓位址為 3 的內容固定為 2'b11,
    Non-Support
    ROM_Data [4] = 2'b00; // 讓位址為 4 的內容固定為 2'b00, `Plus
    ROM_Data [5] = 2'b01; // 讓位址為 5 的內容固定為 2'b01, `Minus
    ROM_Data [6] = 2'b10; // 讓位址為 6 的內容固定為 2'b10, `Bin_And
    ROM_Data [7] = 2'b11; // 讓位址為 7 的內容固定為 2'b11,
    Non-Support
    Data_Out = ROM_Data[Address]; // 由 ROM_Data 內取出資料
end
endmodule

```

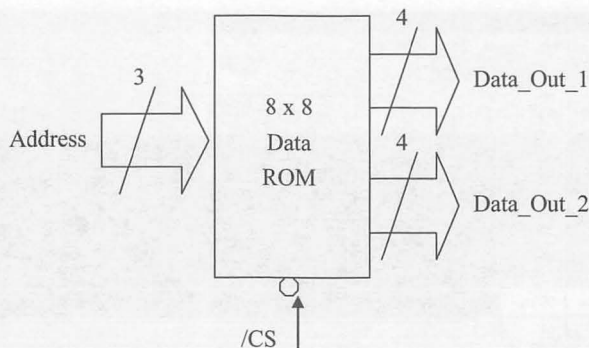


▲ Inst\_ROM.V 電路的模擬結果

**Step3** Data\_ROM.V : 存放 8 個字組，每個字組有 8 個位元的 '資料' 唯讀記憶體

我們將：0x55、0xA5、0xAA、0x00、0xFF、0x0F、0xF0 以及 0x5A 等 8 個字組存放在 Data\_ROM.V 中，作為推動 ALU\_2.V 簡易的算術邏輯運算單元 (ALU) 的資料碼來源。

這顆 ROM 和 Inst\_ROM.V 的 ROM 不同的地方在於：它有二個 4 個位元的資料輸出埠 (Data\_Out\_1[3:0] 及 Data\_Out\_2[2:0])，而 Inst\_ROM.V 的 ROM 只有一個 2 個位元的資料輸出埠 (Data\_Out[1:0])。



▲ Data\_ROM.V：存放 8 個字組，每個字組有 8 個位元的'資料'唯讀記憶體的邏輯符號

【Data\_ROM.V：存放 8 個字組，每個字組有 8 個位元的'資料'唯讀記憶體】

```

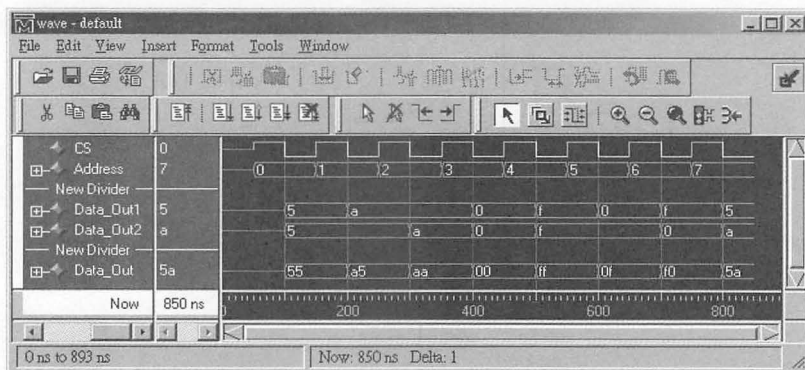
// Data_ROM.V：存放'資料'的唯讀記憶體
// 具有 8 個字組，每個字組有 8 個位元的唯讀記憶體
module Data_ROM (CS, Address, Data_Out1, Data_Out2);
input          CS;
input [2:0] Address;
output [3:0] Data_Out1;
reg [3:0] Data_Out1;
output [3:0] Data_Out2;
reg [3:0] Data_Out2;
// 使用 4 個 8 Bits 的 Register，來模擬 ROM
reg [7:0] ROM_Data[7:0];
reg [7:0] Data_Out;

always @(negedge CS) // 選擇到此晶片
begin
    ROM_Data [0] = 8'b0101_0101; // 讓位址為 0 的內容固定為 0x55
    ROM_Data [1] = 8'b1010_0101; // 讓位址為 1 的內容固定為 0xA5
    ROM_Data [2] = 8'b1010_1010; // 讓位址為 2 的內容固定為 0xAA
    ROM_Data [3] = 8'b0000_0000; // 讓位址為 3 的內容固定為 0x00
    ROM_Data [4] = 8'b1111_1111; // 讓位址為 4 的內容固定為 0xFF
    ROM_Data [5] = 8'b0000_1111; // 讓位址為 5 的內容固定為 0x0F
    ROM_Data [6] = 8'b1111_0000; // 讓位址為 6 的內容固定為 0xF0
    ROM_Data [7] = 8'b0101_1010; // 讓位址為 7 的內容固定為 0x5A
end
    
```

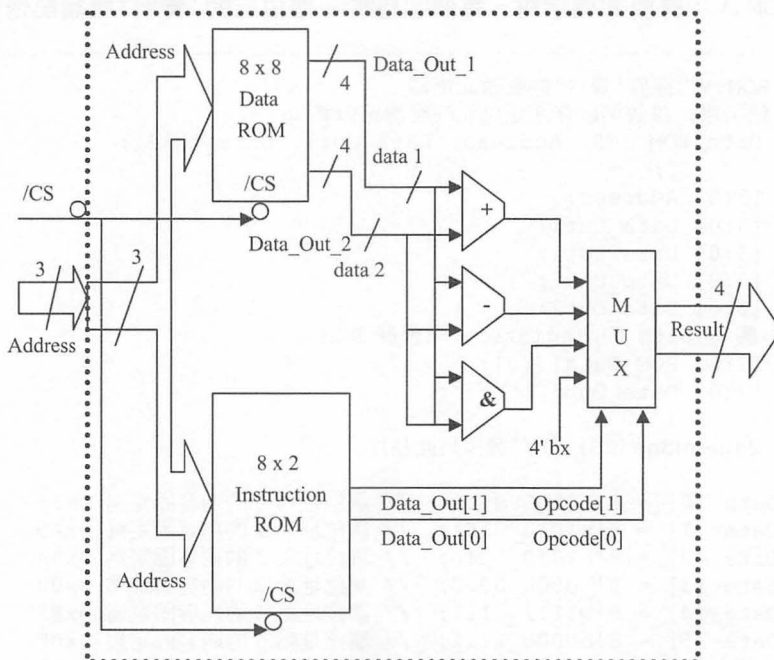
```

Data_Out = ROM_Data[Address]; // 由 ROM_Data 內取出資料
Data_Out1[3:0] = Data_Out[7:4]; // 取出位元 [7:4] 給 Data_Out1
Data_Out2[3:0] = Data_Out[3:0]; // 取出位元 [3:0] 給 Data_Out2
end
endmodule

```



▲ Data\_ROM.V 電路的模擬結果



▲ ROM\_ALU.V：使用一顆指令 ROM 與一顆資料 ROM 來推動特定功能的 ALU 的邏輯符號

【ROM\_ALU.V：使用一顆指令 ROM 與一顆資料 ROM 來推動特定功能的 ALU】

```
// ROM_ALU.V
// 使用一顆指令 ROM 與一顆資料 ROM 來推動特定功能的 ALU
// Inst_ROM.V, 存放 8 個字組, 每個字組有 2 個位元的'指令'唯讀記憶體
// Data_ROM.V, 存放 8 個字組, 每個字組有 8 個位元的'資料'唯讀記憶體
// ALU_2.V, 簡易的算術邏輯運算單元 (ALU)
//

// 引用 Inst_ROM.V, Data_ROM.V 以及 ALU_2.V
`include "Inst_ROM.V" // 8x2 的 指令 ROM (Instruction ROM)
`include "Data_ROM.V" // 8x8 的 資料 ROM (Data ROM)
`include "ALU_2.V" // 簡易的算術邏輯運算單元 (ALU)

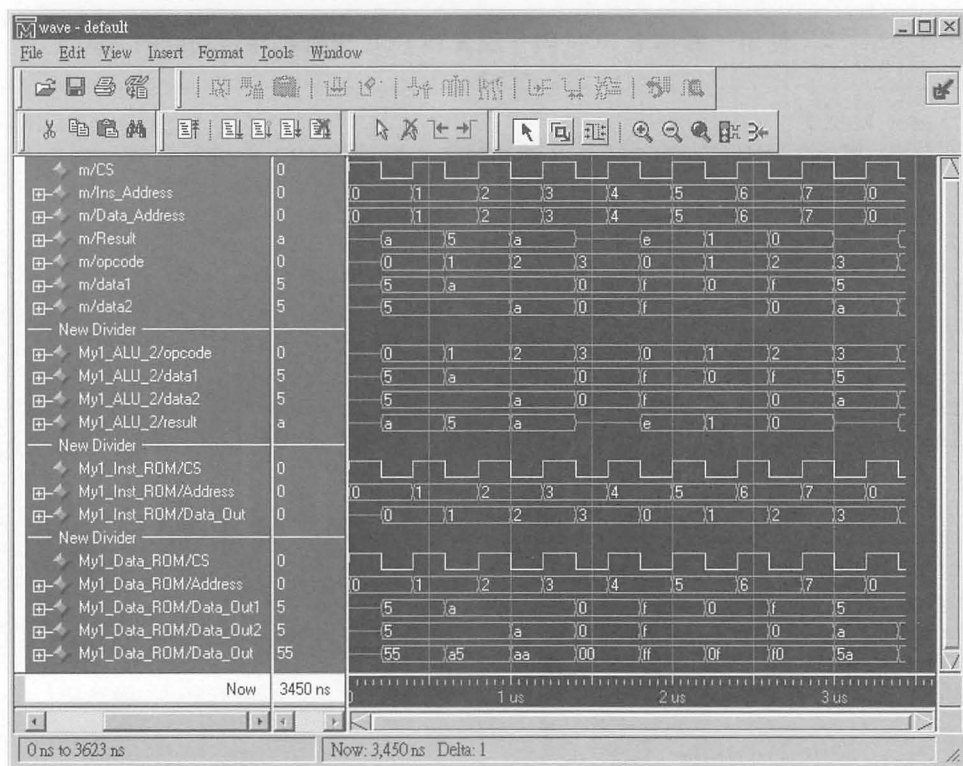
// 具有 32 個字組, 每個字組有 3 個位元的唯讀記憶體
module ROM_ALU (CS, Ins_Address, Data_Address, Result);
input CS;
input [2:0] Ins_Address;
input [2:0] Data_Address;
output [3:0] Result;
wire [3:0] Result;

// ----- 內部的連接線 -----
wire [1:0] opcode;
wire [3:0] data1;
wire [3:0] data2;

// 引用 2 個具有 16 個字組, 每個字組有 3 個位元的唯讀記憶體
Data_ROM My1_Data_ROM (CS, Ins_Address, data1, data2);
Inst_ROM My1_Inst_ROM (CS, Data_Address, opcode);
ALU_2 My1_ALU_2 (opcode, data1, data2, Result);

endmodule
```





▲ ROM\_ALU.V 電路的模擬結果

## 本章練習

### Question 1 :

請比較隨機存取記憶體 (Random Access Memory, RAM) 與 SDRAM、DDR、DDR2、DDR3、GDDR、GDDR2、GDDR3、GDDR4 以及 GDDR5 的差異。

### Question 2 :

請比較唯讀記憶體 (ROM)、EPROM、EEPROM (Double EPROM) 以及快閃記憶體 (Flash) 的差異。

**PS** Google 可以幫您找到解答。